



《AI大模型Ragent项目》——用户说的话 ≠ 该搜的词

来自： 拿个offer-开源&项目实战

 马丁

2026年04月23日 21:52

开篇引言

上一篇把记忆系统的最后一块拼图补上了——摘要压缩。滑动窗口保留最近 8 轮原文，更早的对话浓缩成一段摘要，水位线保证不重复压缩，异步执行不阻塞主流程。到这里，ctx.history 已经装好了完整的对话记忆，Pipeline 的阶段 1 结束。

阶段 2 要做什么？来看一个场景。

假设你在一家电商公司做智能客服助手，接入了 3C 数码、家电、服装等多个品类的商品知识库。有个用户第一轮问了 iPhone 16 Pro 的退货政策是什么，系统从 3C 数码知识库检索到退货政策文档，回答得很好。第二轮，用户追问了一句：

那它的保修期呢？

记忆系统让模型知道 它是 iPhone 16 Pro。但检索引擎呢？检索引擎拿到的查询就是这句原话——那它的保修期呢。拿 它 去向量数据库里搜，能搜到什么？大概率搜到的是各种产品的保修通用条款，可能不是 iPhone 16 Pro 的保修政策。

模型有记忆，但检索没有。这个问题在基础系列的 Query 改写篇已经讲过了——解法是在检索之前把原始问题改写成对检索友好的形式。基础系列讲了五种改写策略的原理，本篇不重复那些概念，聚焦 Ragent 项目里的工程实现。

Ragent 的查询改写有一个特别的设计：一次 LLM 调用同时完成两件事——改写和拆分。改写解决检索精度问题（指代消解、去噪），拆分解决另一个问题——复合问题的意图覆盖。

什么意思？再看一个场景。用户问了一句：

iPhone 16 Pro 的退货政策是什么？ AirPods Pro 的保修期呢？

这一句话里包含两个完全不同方向的问题。如果只改写不拆分，改写后变成 iPhone 16 Pro 退货政策和 AirPods Pro 保修期，拿这一整句话去做意图分类，LLM 可能只匹配到退货政策方向，AirPods Pro 的保修期被忽略了。拆分成两个子问题后，每个子问题独立做意图分类，两个方向都不丢。

本篇就围绕这两件事展开：改写怎么做，拆分怎么做，以及两者怎么在一次调用里同时搞定。

改写 + 拆分的双输出设计

1. 为什么要拆分

改写的价值在基础系列已经讲透了：指代消解、上下文补全、口语化转正式，让检索查询独立、完整、精确。这里重点说拆分。

用具体例子感受一下。用户问 iPhone 16 Pro 的退货政策是什么？ AirPods Pro 的保修期呢：

不拆分的情况：

改写后的完整问题 → "iPhone 16 Pro 退货政策和 AirPods Pro 保修期"（作为唯一子问题）
→ 意图分类 → 可能只命中 kb-return-policy（退货政策节点），AirPods Pro 保修方向被弱化
→ 检索 → 只去退货政策知识库搜相关内容
→ 结果：AirPods Pro 的保修期没人管了

拆分后的情况：

子问题 1 → "iPhone 16 Pro 的退货政策是什么"
→ 意图分类 → 命中 kb-return-policy (score=0.92)

子问题 2 → "AirPods Pro 的保修期"
→ 意图分类 → 命中 kb-warranty (score=0.90)

→ 两个方向都命中，后续分别去对应知识库检索

打个比方，改写像是把一封字迹潦草的信重新写清楚，拆分像是把一封信里问了两件事的信拆成两封，分别交给不同部门处理。

2. 什么时候该拆、什么时候不该拆

不是所有包含多个关键词的问题都该拆。核心判断标准是：子问题能不能独立回答。

场景	是否拆分	原因
iPhone 16 Pro 的退货政策是什么？ AirPods Pro 的保修期呢？	拆分	两个独立问题，指向不同产品的不同主题
iPhone 16 Pro 和 iPhone 16 Plus 的退货政策分别是什么？	拆分	话题相同但指向不同产品，需要分别检索
iPhone 16 Pro 和 iPhone 16 Plus 有什么区别？	不拆分	对比型问题，两个产品放在一起才能回答
iPhone 16 Pro 从哪些方面考虑？	不拆分	笼统询问，没有明确列举子问题
你好	不拆分	问候语，保持原样

对比型问题是最容易误拆的。iPhone 16 Pro 和 iPhone 16 Plus 有什么区别？如果拆成 iPhone 16 Pro 的优缺点和 iPhone 16 Plus 的优缺点两个子问题，分别检索后各自拿到一堆文档，但用户要的是对比，不是两个独立介绍。拆了反而更差。

3. RewriteResult：一个 Record 承载两个输出

改写和拆分的结果统一封装在一个 Java Record 里：

```
public record RewriteResult(String rewrittenQuestion, List<String> subQuestions) {  
    }  
}
```

两个字段各有分工：

rewrittenQuestion：改写后的完整问题。用于歧义引导阶段（阶段 4）展示给用户看，也在不拆分时作为检索查询

subQuestions：子问题列表。每个子问题会独立送去做意图分类（阶段 3）

两者的关系有一个约定：**不拆分时，subQuestions 只有一个元素，内容与 rewrittenQuestion 一致。**这样下游的 IntentResolver 不管拆不拆，都从 subQuestions 取数据，逻辑统一。

术语归一化：LLM 改写前的预处理

在调 LLM 改写之前，Ragent 先做了一步轻量级的规则处理——术语归一化。

1. 解决什么问题

电商平台的产品和品类有很多口语化的叫法。用户习惯说苹果手机，但知识库里的文档标题写的是 iPhone 系列；用户说降噪豆，指的其实是 AirPods Pro。如果不归一化，LLM 改写后可能保留苹果手机这个叫法，后续在 iPhone 相关的知识库里搜苹果手机，向量距离可能偏远，检索精度打折扣。

术语归一化就是在 LLM 改写之前，先用规则把这些口语化叫法映射为标准产品名。

2. QueryTermMappingService 的实现

```
@Service  
@RequiredArgsConstructor  
public class QueryTermMappingService {  
  
    private final QueryTermMappingMapper mappingMapper;  
    private final QueryTermMappingCacheManager cacheManager;  
  
    public String normalize(String text) {  
        if (text == null || text.isEmpty()) {  
            return text;  
        }  
        List<QueryTermMappingDO> mappings = loadMappings();  
        String result = text;  
        for (QueryTermMappingDO mapping : mappings) {  
            result = QueryTermMappingUtil.applyMapping(  
                result, mapping.getSourceTerm(), mapping.getTargetTerm());  
        }  
        return result;  
    }  
}
```

```
}

private List<QueryTermMappingDO> loadMappings() {
    // 优先从 Redis 缓存读取
    List<QueryTermMappingDO> cached = cacheManager.getMappingsFromCache();
    if (CollUtil.isEmpty(cached)) {
        return cached;
    }
    // 缓存未命中，从数据库加载
    List<QueryTermMappingDO> dbList = mappingMapper.selectList(
        Wrappers.lambdaQuery(QueryTermMappingDO.class)
            .eq(QueryTermMappingDO::getEnabled, 1));
    // 按优先级降序 + 源词长度降序排序
    dbList.sort(Comparator
        .comparing(QueryTermMappingDO::getPriority, Comparator.nullsLast(Integer::compareTo)
            .thenComparing(m -> m.getSourceTerm().length(), Comparator.reverseOrder()));
    // 回填 Redis 缓存
    cacheManager.saveMappingsToCache(dbList);
    return dbList;
}
}
```

管理后台增删改映射规则时，Admin 接口先更新新数据库，再删除 Redis 缓存：

```
// QueryTermMappingAdminServiceImpl.java
queryTermMappingMapper.insert(record);    // 先写数据库
queryTermMappingCacheManager.clearCache(); // 再删缓存，下次读自动从 DB 加载
```

这是经典的 Cache-Aside 模式——读时加载、写时失效。和意图树的缓存策略一致，集群部署时任意一台实例的 Admin 操作都会删除 Redis 缓存，所有实例下次 normalize 时发现缓存为空，各自从数据库重新加载，天然保持一致。

几个设计细节：

映射规则缓存在 Redis，而不是本地内存。 管理后台可以在线增删改映射规则（比如新增一条 苹果手机 → iPhone），修改后删除 Redis 缓存，所有实例下次读取时自动加载最新规则，不需要重启应用，也不存在集群节点间数据不一致的问题。

排序策略：优先级高的先替换，同优先级下长词优先。 为什么长词优先？假设有两条规则：苹果 → Apple 和 苹果手机 → iPhone。如果短词苹果先被替换了，原文里的苹果手机就变成了 Apple 手机，长词规则就匹配不到了。长词优先替换，避免这种截断问题。

归一化是纯文本替换，不依赖 LLM。 速度快，零 LLM 开销。相当于在 LLM 改写之前帮它做好功课——把术语问题先解决掉，LLM 只需要专注于指代消解和语义改写。

举个具体的例子：用户输入苹果手机的降噪豆能退吗，归一化后变成 iPhone 的 AirPods Pro 能退吗。这个归一化后的问题再送给 LLM 做进一步改写（去掉口语化表达、补全上下文等）。

一次 LLM 调用搞定改写 + 拆分

1. 核心入口：rewriteWithSplit

先看 Pipeline 怎么调的：

```
// StreamChatPipeline.java 阶段 2
private void rewriteQuery(StreamChatContext ctx) {
    RewriteResult rewriteResult = queryRewriteService.rewriteWithSplit(
        ctx.getQuestion(), ctx.getHistory());
};
ctx.setRewriteResult(rewriteResult);
}
```

输入是用户原始问题 + 对话历史（阶段 1 的输出），输出是 RewriteResult，存入 Context 供下游消费。

再看 MultiQuestionRewriteService 的核心实现：

```
@Override
@RagTraceNode(name = "query-rewrite-and-split", type = "REWRITE")
public RewriteResult rewriteWithSplit(String userQuestion, List<ChatMessage> history) {
    // 开关关闭 → 术语归一化 + 规则拆分
    if (!ragConfigProperties.getQueryRewriteEnabled()) {
        String normalized = queryTermMappingService.normalize(userQuestion);
        List<String> subs = ruleBasedSplit(normalized);
        return new RewriteResult(normalized, subs);
    }

    // 先做术语归一化
    String normalizedQuestion = queryTermMappingService.normalize(userQuestion);

    // 再调 LLM 改写 + 拆分
    return callLLMRewriteAndSplit(normalizedQuestion, userQuestion, history);
}
```

逻辑很清晰：

1. 不管开关开不开，先过一遍术语归一化
2. 开关关闭 → 走纯规则路径（归一化 + 按标点拆分），不调 LLM
3. 开关打开 → 归一化后的问题送去 LLM 改写 + 拆分

2. 消息数组的构建

LLM 调用的消息数组怎么拼？来看 buildRewriteRequest：

```
private ChatRequest buildRewriteRequest(String systemPrompt,
                                         String question,
                                         List<ChatMessage> history) {

    List<ChatMessage> messages = new ArrayList<>();
    messages.add(ChatMessage.system(systemPrompt));

    // 只保留最近 1-2 轮的 User 和 Assistant 消息
    if (CollUtil.isEmpty(history)) {
        List<ChatMessage> filtered = history.stream()
            .filter(msg -> msg.getRole() == ChatMessage.Role.USER
                || msg.getRole() == ChatMessage.Role.ASSISTANT)
            .toList();

        List<ChatMessage> recentHistory = filtered.subList(
            Math.max(0, filtered.size() - 4), filtered.size()); // 最多 4 条 = 2 轮
        messages.addAll(recentHistory);
    }

    messages.add(ChatMessage.user(question));

    return ChatRequest.builder()
        .messages(messages)
        .temperature(0.1D)
        .topP(0.3D)
        .thinking(false)
        .build();
}
```

最终拼出来的消息数组是这样的：

```
[0] SYSTEM:    改写 Prompt（角色、规则、输出格式、示例）
[1] USER:      最近第 N-1 轮用户消息（如有）
[2] ASSISTANT:  最近第 N-1 轮助手回复（如有）
[3] USER:      最近第 N 轮用户消息（如有）
```

```
[4] ASSISTANT: 最近第 N 轮助手回复（如有）
[5] USER:      当前归一化后的问题
```

这里有三个关键的设计决策：

2.1 历史消息只取最近 2 轮

改写的主要目的是指代消解——把 它 这个 替换成具体实体。一般 1~2 轮历史就够了。用户第一轮问了 iPhone 16 Pro 的退货政策，第二轮追问那它的保修期呢，只需要看到第一轮就知道 它 是 iPhone 16 Pro。送太多历史不仅浪费 Token，还可能引入噪音——LLM 看到更早的对话可能做出过度改写，把原始问题改得面目全非。

2.2 过滤掉 SYSTEM 消息

ctx.history 里可能包含摘要（SYSTEM 类型的消息）。摘要 是 给最终生成用的上下文，对改写没有价值——改写只需要看最近的 user/assistant 交互就够了。过滤掉 SYSTEM 消息避免了不必要的 Token 消耗。

2.3 LLM 参数追求确定性

temperature=0.1, topP=0.3, thinking=false。改写是一个确定性任务——同一个问题在同样的上下文下应该改写成同样的结果。不需要创造性，不需要深度思考，要的是稳定可靠。对比一下上一篇摘要压缩的参数（temperature=0.3, topP=0.9）就能看出区别：摘要需要一定的归纳灵活性，改写需要的是精确和一致。

3. Prompt 模板逐块解读

改写的 Prompt 模板 user-question-rewrite.st 是整个改写逻辑的灵魂。它要在一个 Prompt 里同时指导 LLM 完成改写和拆分两件事，还不能乱来。来看它的完整结构：

3.1 角色和输出格式

```
# 角色
你是查询改写助手，用于 RAG 检索阶段。

# 任务
1. 将用户问题改写成适合检索的自然语言查询
2. 判断是否需要拆分成多个子问题

# 输出格式
严格返回 JSON，不要额外文字：
{
  "rewrite": "改写后的查询",
  "should_split": true/false,
  "sub_questions": ["子问题1", "子问题2"]
}
```

三个字段各有用途：rewrite 对应 RewriteResult.rewrittenQuestion，sub_questions 对应 RewriteResult.subQuestions，should_split 是辅助判断字段，帮助 LLM 明确自己的决策。用 JSON 格式输出而不是纯文本，是为了方便程序解析。如果只要求 LLM 返回改写后的文本，就没法同时输出子问题列表了。

3.2 改写规则

```
# 改写规则

## 保留内容
- 专有名词（系统名、产品名、模块名等）：原样保留，不得修改
- 关键限制：时间范围、环境、终端类型、角色身份等
- 业务场景：流程、规范、配置等

## 删除内容
- 礼貌用语："请帮我"、"麻烦"、"谢谢"
- 回答指令："详细说明"、"分点回答"、"一步步分析"
- 无关描述："我是新人"、"我刚入职"

## 禁止行为
- 不得添加原文没有的条件、维度、假设
- 不得修改专有名词的写法
```

- 不得引入 方面/维度/角度 等枚举词（除非原文有）
- 保持原问题的语言（中文/英文）

这三组规则划定了改写的边界：

保留什么： 专有名词和关键限制是检索的核心信号。iPhone 16 Pro 在京东自营的退货政策这句话里，iPhone 16 Pro、京东自营、退货政策每一个词都可能命中不同的 chunk。改掉了就搜不到。

删除什么： 请帮我详细介绍一下这类礼貌用语和回答指令对检索毫无价值，反而引入噪音。拿这些词一起做向量化，会干扰核心关键词的语义匹配。

禁止什么： 这条最容易被忽视，也最容易出问题。LLM 有时候会过度改写——你说 iPhone 16 Pro 的退货政策，它给你改成 iPhone 16 Pro 的退货政策，包括七天无理由、拆封后退货、运费承担等环节。多出来的枚举词是 LLM 自己加的，不是用户说的，可能把检索引到错误的方向。所以要明确禁止添加原文没有的内容。

3.3 拆分规则

拆分规则

何时拆分

只在以下情况拆分：

- 多个问号："系统A怎么用？系统B呢？"
- 显式列举："1) ... 2) ...","A和B分别是什么？"
- 分号/换行分隔

何时不拆分

- 抽象对比："X和Y有什么区别？" → 不拆分
- 笼统询问："从哪些方面考虑？" → 不拆分（没有具体列举）
- 不确定时 → 不拆分

一致性约束

- 若不拆分：sub_questions 只包含 1 条，且与 rewrite 完全一致
- 若拆分：每个子问题尽量保持原文表述

拆分规则的心理理念是**宁可拆也不要错拆**。不确定时不拆分这条兜底规则很重要——错误的拆分比不拆分危害更大。把一个对比型问题拆成两个独立问题，后续检索出来的结果各说各的，没有对比，用户拿到的答案反而更差。

一致性约束保证了下游的统一处理。不管 LLM 决定拆不拆，sub_questions 数组里始终有内容，IntentResolver 直接遍历 sub_questions 就行，不需要判断到底拆没拆。

针对不同的系统场景，问题改写的提示词可能是不同的，当前项目中算是一种中间值。

3.4 示例的选择

Prompt 里附了 5 个示例，每个覆盖一种核心场景：

示例	场景	要点
示例 1	删除礼貌用语	请帮我详细介绍一下 → 删掉，不拆分
示例 2	保留专有名词和限制	OA 系统在移动端的 → 全部保留，不拆分
示例 3	拆分多问句	两个问号 → 拆成两个子问题
示例 4	不拆分抽象对比	X 和 Y 有什么区别？ → 不拆分
示例 5	指代消解	结合历史把 它 替换成具体实体，不拆分

五个示例刚好覆盖了改写和拆分的关键决策边界。示例 3 和示例 4 放在一起尤其有意义——一个该拆一个不该拆，让 LLM 看到区分标准。

4. JSON 解析与容错

LLM 返回的 JSON 不能完全信任。有时候会多包一层 Markdown 代码块标记（``json ... ``），有时候 sub_questions 是空数组，有时候连 rewrite 字段都没有。parseRewriteAndSplit 方法处理了这些边界情况：

```
private RewriteResult parseRewriteAndSplit(String raw) {
    try {
        // 第 1 层容错：移除 Markdown 代码块标记
        String cleaned = LLMResponseCleaner.stripMarkdownCodeFence(raw);
```

```
JsonElement root = JsonParser.parseString(cleaned);
if (!root.isJsonObject()) return null;

JsonObject obj = root.getAsJsonObject();
String rewrite = obj.has("rewrite") ? obj.get("rewrite").getString().trim() : "";

// 第 2 层容错：逐个元素校验子问题数组
List<String> subs = new ArrayList<>();
if (obj.has("sub_questions") && obj.get("sub_questions").isArray()) {
    JSONArray arr = obj.getAsJSONArray("sub_questions");
    for (JsonElement el : arr) {
        if (el.isJsonPrimitive() && el.getAsJsonPrimitive().isString()) {
            String s = el.getString().trim();
            if (StringUtil.isNotBlank(s)) {
                subs.add(s);
            }
        }
    }
}

// 第 3 层容错：rewrite 为空则整体失败
if (StringUtil.isBlank(rewrite)) return null;

// 第 4 层容错：没有子问题则用 rewrite 兜底
if (CollUtil.isEmpty(subs)) {
    subs = List.of(rewrite);
}

return new RewriteResult(rewrite, subs);
} catch (Exception e) {
    log.warn("解析改写+拆分结果失败，raw={}", raw, e);
    return null;
}
}
```

LLMResponseCleaner.stripMarkdownCodeFence 的实现也值得一看：

```
private static final Pattern LEADING_CODE_FENCE = Pattern.compile("`[\w-]*\s*\n?");
private static final Pattern TRAILING_CODE_FENCE = Pattern.compile("\n?```\s*$");

public static String stripMarkdownCodeFence(String raw) {
    if (raw == null) return null;
    String cleaned = raw.trim();
    cleaned = LEADING_CODE_FENCE.matcher(cleaned).replaceFirst("");
    cleaned = TRAILING_CODE_FENCE.matcher(cleaned).replaceFirst("");
    return cleaned.trim();
}
```

用正则移除头尾的 `` 标记，支持带语言标识的代码块（比如 ``json）。这个工具方法在项目里多处复用，凡是需要从 LLM 返回中提取 JSON 的地方都会先过一遍。

和 LLM 打交道久了就会明白一个道理：你在 Prompt 里写了严格返回 JSON、不要额外文字，LLM 大概率还是会时不时给你包一层 Markdown。不能在解析端假设 LLM 一定遵守格式要求，每个边界都要有容错。

接口设计：三层方法，逐级降级

QueryRewriteService 接口的设计体现了一种渐进式降级的思路：

```
public interface QueryRewriteService {
```

```
/** 纯改写，最基础的能力 */
String rewrite(String userQuestion);

/** 改写 + 拆分（无历史版本） */
default RewriteResult rewriteWithSplit(String userQuestion) {
    String rewritten = rewrite(userQuestion);
    return new RewriteResult(rewritten, List.of(rewritten));
}

/** 改写 + 拆分 + 历史上下文（完整版本） */
default RewriteResult rewriteWithSplit(String userQuestion, List<ChatMessage> history) {
    return rewriteWithSplit(userQuestion);
}
}
```

三个方法从简单到完整：rewrite → rewriteWithSplit → rewriteWithSplit(question, history)。后两个都有 default 实现，往上一级回退。

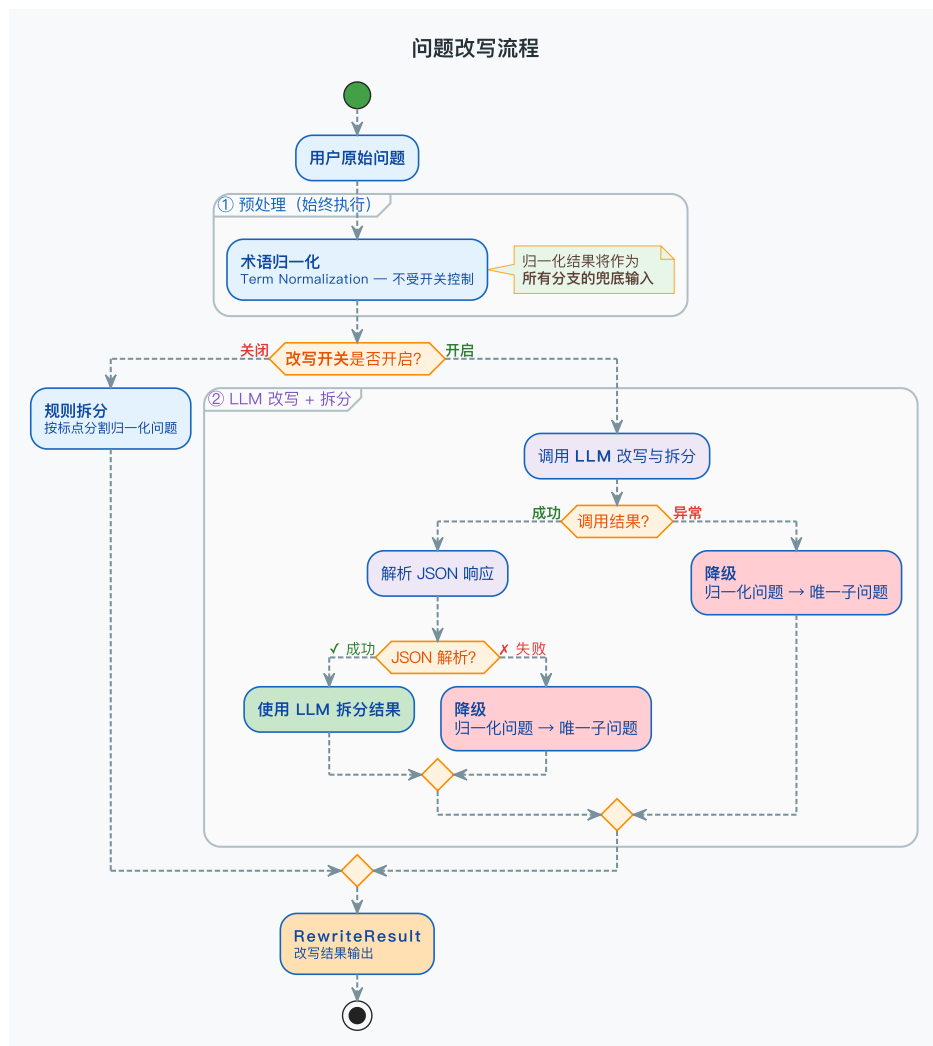
这意味着如果有一天你想换一个简单的改写实现——比如一个纯规则的改写器，只覆盖 rewrite 方法就行了。default 实现会自动把改写结果包装成 RewriteResult（改写结果作为唯一子问题），Pipeline 调用 rewriteWithSplit(question, history) 时一路回退到 rewrite，不会报错。

Pipeline 调用的是最完整的版本 rewriteWithSplit(question, history)，MultiQuestionRewriteService 直接覆盖了这个方法，用 LLM 一次调用同时完成改写和拆分。rewrite 和无历史的 rewriteWithSplit 目前没有被实现——它们是为将来扩展预留的，比如哪天想换一个纯规则的改写器，只覆盖 rewrite 就够了，default 实现会自动兜住上层调用。

兜底策略：改写关了怎么办、调用失败怎么办

和记忆系统的摘要压缩一样，改写也是一个增强项。改写失败不应该中断整个问答流程，最坏的情况就是拿原始问题去检索，精度差一点。

1. 三层兜底链路



用表格总结：

异常场景	兜底策略	结果
改写开关关闭	术语归一化 + 规则拆分	RewriteResult (归一化问题 + 按标点拆分的子问题)
LLM 调用异常	归一化问题作为改写结果和唯一子问题	RewriteResult (归一化问题, [归一化问题])
JSON 解析失败	同上	同上

来看兜底逻辑的代码：

```
private RewriteResult callLLMRewriteAndSplit(String normalizedQuestion,
                                             String originalQuestion,
                                             List<ChatMessage> history) {
    String systemPrompt = promptTemplateLoader.load(QUERY_REWRITE_AND_SPLIT_PROMPT_PATH);
    ChatRequest req = buildRewriteRequest(systemPrompt, normalizedQuestion, history);

    try {
        String raw = llmService.chat(req);
        RewriteResult parsed = parseRewriteAndSplit(raw);

        if (parsed != null) {
            log.info("查询改写+拆分: 原始={}, 归一化={}, 改写={}, 子问题={}",
                    originalQuestion, normalizedQuestion,
                    parsed.rewrittenQuestion(), parsed.subQuestions());
            return parsed;
        }

        log.warn("解析失败, 使用归一化问题兜底");
    } catch (Exception e) {
        log.warn("LLM 调用失败, 使用归一化问题兜底", e);
    }

    // 统一兜底: 归一化问题作为改写结果和唯一子问题
    return new RewriteResult(normalizedQuestion, List.of(normalizedQuestion));
}
```

注意兜底用的是 normalizedQuestion 而不是原始的 userQuestion。即使 LLM 挂了，术语归一化的成果不会白费——苹果手机至少已经被映射成 iPhone 了。

2. 规则拆分：改写关闭时的兜底方案

改写开关关闭时，没有 LLM 参与，拆分靠纯规则——按标点符号分割：

```
private List<String> ruleBasedSplit(String question) {
    List<String> parts = Arrays.stream(question.split("[? 。; ;\\n]+"))
        .map(String::trim)
        .filter(StrUtil::isNotBlank)
        .collect(Collectors.toList());

    if (CollUtil.isEmpty(parts)) {
        return List.of(question);
    }

    // 分割符包含 ? 和 ? , 分割后片段末尾不会再有问号，统一补上
    return parts.stream()
        .map(s -> s + "? ")
        .toList();
}
```

按问号、句号、分号、换行符拆分。因为问号本身是分隔符，分割后片段末尾不会保留问号，所以统一给每个片段补上问号，保持问句形式。简单粗暴，但在改写开关关闭的场景下，至少能覆盖最基本的多问句情况——两个问号分隔的两个问题。
拆分结果为空时，回退为原问题作为唯一子问题，保证返回值永远不为空。

RewriteResult 怎么喂给下游

改写和拆分完成后，RewriteResult 存入 ctx.rewriteResult。下一站是阶段 3——意图识别。来看 IntentResolver 怎么消费这个结果。

1. IntentResolver.resolve 的入口

```
@RagTraceNode(name = "intent-resolve", type = "INTENT")
public List<SubQuestionIntent> resolve(RewriteResult rewriteResult) {
    // 取子问题列表；如果为空，回退到改写后的完整问题
    List<String> subQuestions = CollUtil.isEmpty(rewriteResult.subQuestions())
        ? rewriteResult.subQuestions()
        : List.of(rewriteResult.rewrittenQuestion());

    // 每个子问题并行做意图分类
    List<CompletableFuture<SubQuestionIntent>> tasks = subQuestions.stream()
        .map(q -> CompletableFuture.supplyAsync(
            () -> {
                try {
                    return new SubQuestionIntent(q, classifyIntents(q));
                } catch (Exception e) {
                    log.error("子问题意图分类失败，降级为空意图，question: {}", q, e);
                    return new SubQuestionIntent(q, List.of());
                }
            },
            intentClassifyExecutor
        ))
        .toList();

    List<SubQuestionIntent> subIntents = tasks.stream()
        .map(CompletableFuture::join)
        .toList();

    return capTotalIntents(subIntents);
}
```

核心逻辑：

- 1. 从 RewriteResult 取 subQuestions 列表。如果为空（理论上不会，但防御性编程），回退到 rewrittenQuestion 作为唯一子问题
- 2. 每个子问题通过 CompletableFuture 在专用线程池 intentClassifyExecutor 上并行做意图分类
- 3. 每个子问题的分类结果封装为 SubQuestionIntent（子问题文本 + 该子问题命中的意图节点列表）
- 4. 最后经过 capTotalIntents 封顶算法控制总意图数——防止子问题太多、每个又命中多个意图，导致后续检索开销爆炸

classifyIntents 的内部实现和 capTotalIntents 封顶算法是第 5~7 篇的内容，本篇只需要知道：每个子问题独立分类，分类结果是一个带分数的意图节点列表。

2. 用一个完整例子串联全链路

从用户输入到意图分类完成，走一遍完整的数据流：

输入：请帮我详细介绍一下苹果手机的退货流程？降噪豆的保修期呢？

```
Step 1: 术语归一化
"苹果手机" -> "iPhone", "降噪豆" -> "AirPods Pro"
归一化后 -> "请帮我详细介绍一下iPhone的退货流程？ AirPods Pro的保修期呢？ "
```

```
Step 2: LLM 改写 + 拆分
消息数组 = [System Prompt, （最近2轮历史，如有），归一化后的问题]
LLM 返回 -> {
```

```
"rewrite": "iPhone退货流程和AirPods Pro保修期",
"should_split": true,
"sub_questions": ["iPhone的退货流程", "AirPods Pro的保修期"]
}
```

Step 3: 解析为 RewriteResult

```
rewrittenQuestion = "iPhone退货流程和AirPods Pro保修期"
subQuestions = ["iPhone的退货流程", "AirPods Pro的保修期"]
```

Step 4: IntentResolver.resolve (阶段 3)

子问题 1 "iPhone的退货流程"

→ 并行意图分类 → 命中 kb-return-policy (score=0.92)

子问题 2 "AirPods Pro的保修期"

→ 并行意图分类 → 命中 kb-warranty (score=0.89)

→ 两个方向都命中，后续分别去对应知识库检索

对比一下如果不拆分：

改写后 "iPhone退货流程和AirPods Pro保修期" 作为唯一子问题

→ 意图分类 → 可能只命中 kb-return-policy (score=0.78)

→ AirPods Pro 的保修期方向被漏掉

拆分的核心价值就在这里：把一个复合问题变成多个独立问题，让每个问题都能精确命中对应的意图节点。

还要注意一个细节——术语归一化在拆分之前就生效了。如果没有归一化，LLM 改写可能保留降噪豆这个叫法，后续意图分类在 AirPods Pro 相关的意图节点里搜降噪豆就可能匹配不上。归一化把别名问题提前解决了，LLM 只需要专注于语义层面的改写和拆分。

特殊场景处理

1. 问候语和身份类问题

Prompt 模板里有一条特殊规则：

```
## 特殊场景
- 问候/身份类问题 ("你好"、"你是谁")：保持原样
```

用户说你好，改写成什么？不改，原样输出。这类问题在阶段 3 会被识别为 SYSTEM 意图，在阶段 5 直接系统直答，根本不走检索。改写它毫无意义。

2. 单轮对话（无历史）

第一轮对话时 history 为空或者只有摘要（SYSTEM 消息），过滤后没有 user/assistant 消息。这种情况下消息数组只有 System Prompt + 当前问题，LLM 没有历史可参考，自然不会做指代消解——因为没有指代需要消解。

Prompt 里的五个示例中有四个是无历史场景，只有示例 5 是带历史的指代消解。这个分布告诉 LLM：大部分情况下不需要做指代消解，只需要做去噪和判断是否拆分。

3. 改写后问题和原始问题一致

有时候用户的问题已经足够清晰完整，不需要任何改写。Prompt 里写了如果问题已经足够完整清晰请原样输出，LLM 会返回和输入一样的文本。

这种情况没有问题——RewriteResult 的 rewrittenQuestion 和原始问题一样，subQuestions 也只有一个元素就是原始问题本身。下游的处理逻辑完全一致，不需要特判。

配置项

配置项	默认值	含义	调优建议
rag.query-rewrite.enabled	true	改写功能开关	开发调试时可以关闭减少 LLM 调用；生产环境建议开

改写用的模型不需要太强。Ragent 用的是通用的 llmService.chat，实际生产中可以考虑用轻量级模型做改写（比如 7B、14B 的模型就够了），把大模型留给最终的答案生成。改写的输入短、输出也短（一个 JSON），轻量模型完全胜任。

小结

回顾一下本篇的核心要点：

- 1. 一次 LLM 调用同时完成改写和拆分，输出一个 JSON 包含 rewrite 和 sub_questions，封装为 RewriteResult（改写后的完整问题 + 子问题列表）
- 2. 子问题拆分解决复合问题的意图覆盖问题——不拆分只能命中一个方向，拆分后每个子问题独立做意图分类，多个方向都不丢
- 3. LLM 改写之前先经过术语归一化——纯规则的文本替换，零 LLM 开销，把别名映射为标准术语，帮 LLM 做好功课
- 4. 三层兜底保证链路不中断——开关关闭走规则拆分，LLM 调用失败用归一化问题兜底，JSON 解析失败也兜底
- 5. RewriteResult 的子问题列表直接喂给 IntentResolver，每个子问题在专用线程池上并行做意图分类

子问题列表生成好了，下一步就是意图分类——判断每个子问题应该去哪个知识库找答案、调哪个工具、还是系统直接回答。但基础系列里学的四分类（知识检索/工具调用/闲聊/引导澄清）够用吗？假设电商平台接入了 3C 数码、家电、服装、售后政策、物流配送五六个知识库，四分类只能告诉你该查知识库，但查哪个？下一篇讲意图树——四分类撑不住 20 个知识库，为什么要设计意图树？

